DAM
Desarrollo de Aplicaciones Multiplataforma
2º Curso

AD
Acceso a Datos

UD 3
Manejo de Conectores (JDBC)
Parte 2

IES BALMIS
Dpto Informática
Curso 2026-2027
Versión 1 (09/2026)

UD3 – Manejo de conectores

ÍNDICE

1. Introducción
2. Protocolos de acceso a bases de datos
3. Conexión a una base de datos
4. Operaciones: ejecución de consultas
5. Conexión JDBC a MySQL
6. Sentencias de definición de datos
7. Sentencias de manipulación de datos
8. Ejecución de procedimientos
9. Gestión de errores
10. MetaData
11. Otros SGBD de tipo BDR
12. Uso de formatos en las BDR

5. Conexión JDBC a MySQL

Tenemos una BD en MySQL con el nombre **biblioteca** y en ella la siguiente tabla libros con 7 registros:

```
CREATE SCHEMA biblioteca DEFAULT CHARACTER SET utf8mb4
                         COLLATE utf8mb4_general_ci;

USE biblioteca ;

CREATE TABLE libros (
      id          INT PRIMARY KEY AUTO_INCREMENT,
      titulo      VARCHAR(60),
      autor       VARCHAR(60)
);

INSERT INTO libros ( titulo, autor ) VALUES
  ('Macbeth', 'William Shakespeare'),
  ('La Celestina (Tragicomedia de Calisto y Melibea)', 'Fernando de Rojas'),
  ('El Lazarillo de Tormes', 'Anónimo'),
  ('20.000 Leguas de Viaje Submarino', 'Julio Verne'),
  ('Alicia en el País de las Maravillas', 'Lewis Carrol'),
  ('Cien Años de Soledad', 'Gabriel García Márquez'),
  ('La tempestad', 'William Shakespeare');
```

Ahora vamos a añadir una restricción para que no pueda haber un título repetido:

```
USE biblioteca ;

CREATE UNIQUE INDEX ititulo ON libros(titulo);
```

A nivel genérico, los pasos necesarios serán:

- Abrir una conexión.
- Preparar un objeto de tipo "**Statement**", que representará a una (o varias) sentencia(s) SQL.
- A través de ese "Statement", lanzar consultas y obtener resultados.
- Usar excepciones para gestionar los posibles errores.

Ya como aplicación real, los pasos que emplearemos para conectar y obtener un bloque de información son los siguientes:

- Usar "**Class.forName**" para cargar el driver que se haya escogido. En nuestro caso será:

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

- Establecer una conexión, indicando la URL del servidor, el usuario que conecta y la contraseña. Será algo como:

```
Connection con = DriverManager.getConnection(url, usuario, password);
```

- Preparar una sentencia que se ejecutará sobre esa conexión:

```
Statement statement = con.createStatement();
```

- La orden SQL concreta a ejecutar se indicará en un método "**executeQuery**". Si se trata de una consulta que devuelva datos, éstos estarán accesibles a través de un "conjunto de resultados" ("**ResultSet**"):

```
ResultSet rs = statement.executeQuery("SELECT id, titulo, autor FROM libros");
```

- Ese conjunto de resultados se recorrerá de forma secuencial: "**rs.next()**" será verdadero si hay más datos en ese conjunto de resultados:

```
while (rs.next()) {
  ...
}
```

Si cada fila del resultado está formada por varios campos, podemos obtener el valor de cada uno de ellos con "**rs.getString(x)**", donde x puede ser un número (la posición de esa columna en la consulta, comenzando con 1) o un nombre de campo (indiferente de mayúsculas o minúsculas):

```
String titulo  = rs.getString("titulo");
String autor   = rs.getString(3);
```

(si un campo es numérico, no hace falta leerlo como cadena y posteriormente convertirlo a número, sino que se puede emplear otros getters como "**.getInt(1)**".

- Finalmente, se deberá cerrar el ResultSet, el Statement y la conexión:

```
rs.close();
statement.close();
con.close();
```

En nuestros primeros ejemplos no haremos ningún control de errores, nos limitaremos a permitir que "**main**" lance la correspondiente excepción (lo que provocaría el final de la ejecución).

Así, un ejemplo completo podría ser:

JDBC01Select

```
public class JDBC01Select {
  public static void main(String[] args) throws ClassNotFoundException, SQLException {

    // Conexión a la BD
    String url;
    url = "jdbc:mysql://localhost:3306/biblioteca";
    url +="?autoReconnect=true&useSSL=false&zeroDateTimeBehavior=convertToNull";
    url +="&serverTimezone=UTC";
    String usuario = "root";
    String password = "1234";

    String sentenciaSQL="SELECT id, titulo, autor FROM libros";

    Class.forName("com.mysql.cj.jdbc.Driver");
    Connection con = DriverManager.getConnection(url, usuario, password);

    // Crear Statement de la Consulta
    Statement statement = con.createStatement();

    // Resulset
    ResultSet rs = statement.executeQuery(sentenciaSQL);
    System.out.printf("%-60s %-60s\n", "Título", "Autor");
    System.out.print("------------------------------------------------------------");
    System.out.print(" ");
    System.out.print("------------------------------------------------------------");
    System.out.println();
    while (rs.next()) {
      System.out.printf("%-60s %-60s\n", rs.getString("titulo"), rs.getString("autor"));
    }

    // Cerrar objetos
    rs.close();
    statement.close();
    con.close();
  }
}
```

Errores

(En un caso real, es habitual que en vez de "**localhost**" se indique la dirección IP del servidor, que no tiene por qué coincidir con el cliente, por ejemplo 10.100.0.1). Es posible que al lanzar este programa obtengamos el mensaje de error:

```
Exception in thread "main" java.lang.ClassNotFoundException:
org.mysql.jdbc.Driver at ...
```

porque ese fichero (el driver JDBC de MySQL) no se encuentre en el "class path" (la ruta de búsqueda para clases auxiliares del programa), por lo que recuerda añadirlo en la carpeta lib como se explicará a continuación.

Compilar y lanzar aplicación desde consola

Si se compila o se lanza desde el terminal, es posible indicar la ruta del driver JDBC usando la opción "-cp" (o "-classpath"):

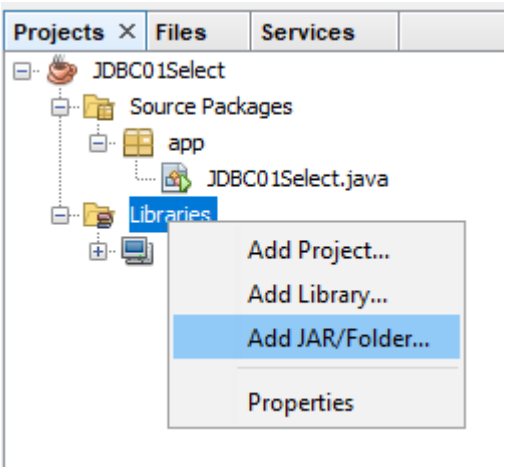
```
javac -cp mysql-connector-java-X.X.X-bin.jar; JDBC01Select.java
java  -cp mysql-connector-java-X.X.X-bin.jar; JDBC01Select
```

Netbeans

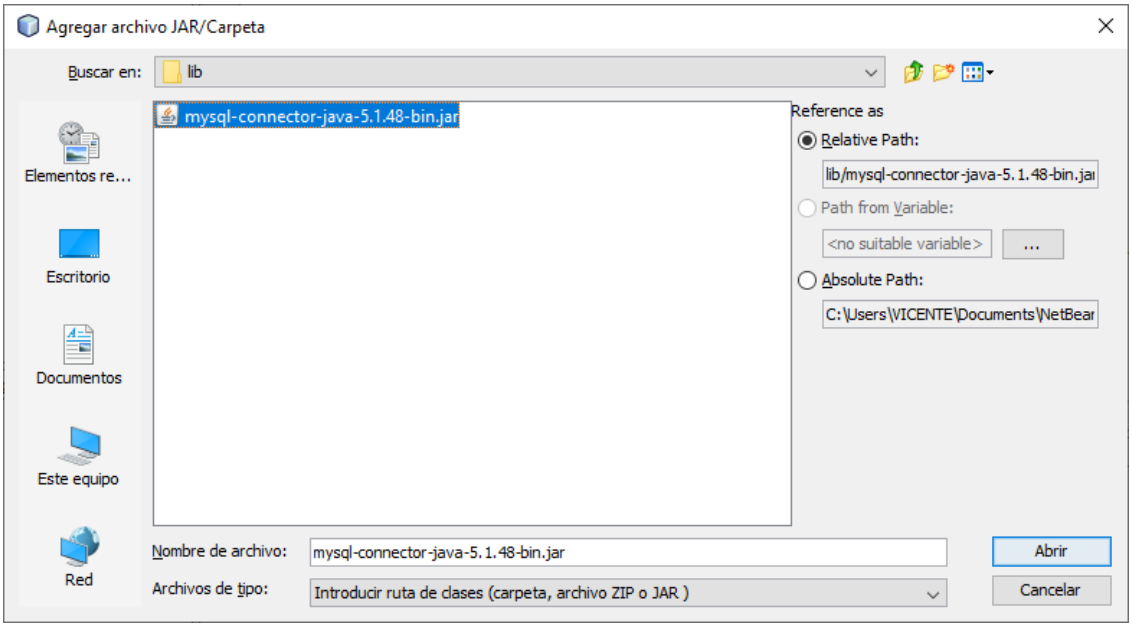
Desde NetBeans también se podrá crear con facilidad proyectos que accedan a bases de datos.

La forma de añadir una biblioteca a un proyecto es pulsar el botón derecho sobre el proyecto para acceder a sus "**Bibliotecas**" del proyecto.

Y ahí tendremos una opción "**Agregar archivo JAR/Carpeta**".



Crearemos una carpeta denominada **lib** en nuestro proyecto y la añadiremos con path relativo, para que el proyecto sea portable.



Todos los archivos **jar** incluidos en esta carpeta se añadirán al proyecto. En nuestro ejemplo, añadiremos "**mysql-connector-java-X.X.X-bin.jar**".

Una vez esa biblioteca esté añadida al proyecto, éste ya debería compilar y funcionar correctamente.

6. Sentencias de definición de datos

Hemos visto que se pueden lanzar consultas que devuelvan datos con "**.executeQuery**". Si se trata de consultas de definición de datos (parte del DDL, Data Definition Language), como CREATE, ALTER o DROP, se deberá emplear "**.executeUpdate**". Ejemplo:

JDBC02Create

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;

public class JDBC02Create {
  public static void main(String[] args)
                    throws ClassNotFoundException, SQLException {

    // Conexión a la BD
    String url;
    url = "jdbc:mysql://localhost:3306/biblioteca";
    url +="?autoReconnect=true&useSSL=false&zeroDateTimeBehavior=convertToNull";
    url +="&serverTimezone=UTC";

    String usuario = "root";
    String password = "1234";

    Class.forName("com.mysql.cj.jdbc.Driver");
    Connection con = DriverManager.getConnection(url, usuario, password);

    // Crear Statement del CREATE TABLE
    String sentenciaSQL = "CREATE TABLE personas ( "+
                "codigo VARCHAR(4) PRIMARY KEY, "+
                "nombre VARCHAR(50), "+
                "email VARCHAR(40) "+ ");";
    Statement statement = con.createStatement();

    // Execute
    statement.executeUpdate(sentenciaSQL);
    statement.close();

    System.out.println("Tabla personas creada");

    // Cerrar conexión
    con.close();
  }
}
```

7. Sentencias de manipulación de datos

Al igual que para las sentencias SQL de definición de datos, las de manipulación de datos (parte del DML, como INSERT, UPDATE o DELETE) no devolverán un conjunto de datos, y se deberán llamar usando "**.executeUpdate**". Eso sí, en este caso puede merecer la pena comprobar el valor devuelto, que es un número entero, que representa la cantidad de filas afectadas (y que en el caso de las sentencias de definición de datos se podía ignorar, porque siempre vale cero).

JDBC03Insert

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;

public class JDBC03Insert {
  public static void main(String[] args)
                    throws ClassNotFoundException, SQLException {

    // Conexión a la BD
    String url;
    url = "jdbc:mysql://localhost:3306/biblioteca";
    url +="?autoReconnect=true&useSSL=false&zeroDateTimeBehavior=convertToNull";
    url +="&serverTimezone=UTC";

    String usuario = "root";
    String password = "1234";

    Class.forName("com.mysql.cj.jdbc.Driver");
    Connection con = DriverManager.getConnection(url, usuario, password);

    // Crear Statement del Insert
    String sentenciaSQL = "INSERT INTO libros (titulo, autor) VALUES "+
              "('La metamorfosis', 'Franz Kafka'), "+
              "('Romeo y Julieta','William Shakespeare');";
    Statement statement = con.createStatement();

    // Execute
    int cantidad = statement.executeUpdate(sentenciaSQL);
    statement.close();

    System.out.println("Datos insertados: " + cantidad);


    // Cerrar conexión
    con.close();
  }
}
```

La salida de este programa debería ser:

```
Datos insertados: 2
```

Y si ahora volvemos a lanzar el programa **JDBC01Select**, que mostraba todos los datos, deberían mostrarse 9.

```
Titulo                                                 Autor
------------------------------------------------------ ------------------------------
Macbeth                                                William Shakespeare
La Celestina (Tragicomedia de Calisto y Melibea)       Fernando de Rojas
El Lazarillo de Tormes                                 Anónimo
...
La metamorfosis                                        Franz Kafka
Romeo y Julieta                                        William Shakespeare
```

La otra sentencia que forma parte del DML es SELECT, que, como ya hemos visto anteriormente, sí devuelve en general un conjunto de datos, y se lanza con "**.executeQuery**".

Otra opción a la hora de realizar consultas es la utilización de Sentencias preparadas para luego indicar el valor de los parámetros. Para esto utilizaremos los objetos "**PreparedStatement**".

JDBC04Prepare

```
public class JDBC04Prepare {
  public static void main(String[] args)
                    throws ClassNotFoundException, SQLException {
    // Conexión a la BD
    String url;
    url = "jdbc:mysql://localhost:3306/biblioteca";
    url +="?autoReconnect=true&useSSL=false&zeroDateTimeBehavior=convertToNull";
    url +="&serverTimezone=UTC";

    String usuario = "root";
    String password = "1234";

    Class.forName("com.mysql.cj.jdbc.Driver");
    Connection con = DriverManager.getConnection(url, usuario, password);

    // Crear Statement de la Consulta
    String sentenciaSQL = "SELECT titulo, autor FROM libros WHERE autor = ?";
    PreparedStatement pStatement = con.prepareStatement(sentenciaSQL);
    pStatement.setString(1, "William Shakespeare");

    // Resulset
    ResultSet rs = pStatement.executeQuery();
    System.out.printf("%-60s %-60s\n", "Título", "Autor");
    System.out.print("------------------------------------------------------------");
    System.out.print(" ");
    System.out.print("------------------------------------------------------------");
    System.out.println();
    while (rs.next()) {
      System.out.printf("%-60s %-60s\n", rs.getString("titulo"), rs.getString("autor"));
    }

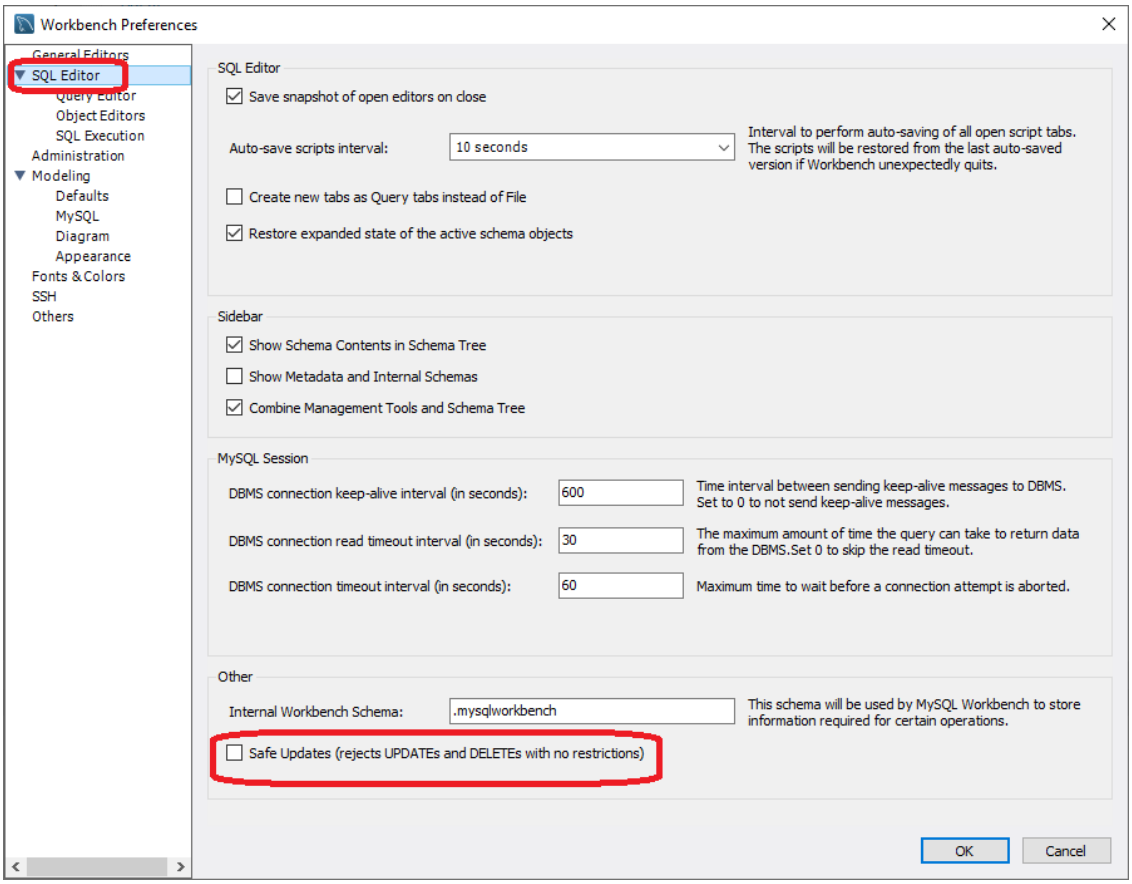
    // Cerrar objetos
    rs.close();
    pStatement.close();
    con.close();
  }
}
```

8. Ejecución de procedimientos

En varias BD como Oracle, MySQL y PostgreSQL se permite definir funciones usando su lenguaje de programación propio como el PL/SQL.

En MySQL, para poder crear procedimientos y funciones sin restricciones, deberemos realizar dos cambios en la configuración:

1) En Workbench, accederemos a "**Edit → Preferences**" y vamos a desactivar el control seguro de actualizaciones de datos:

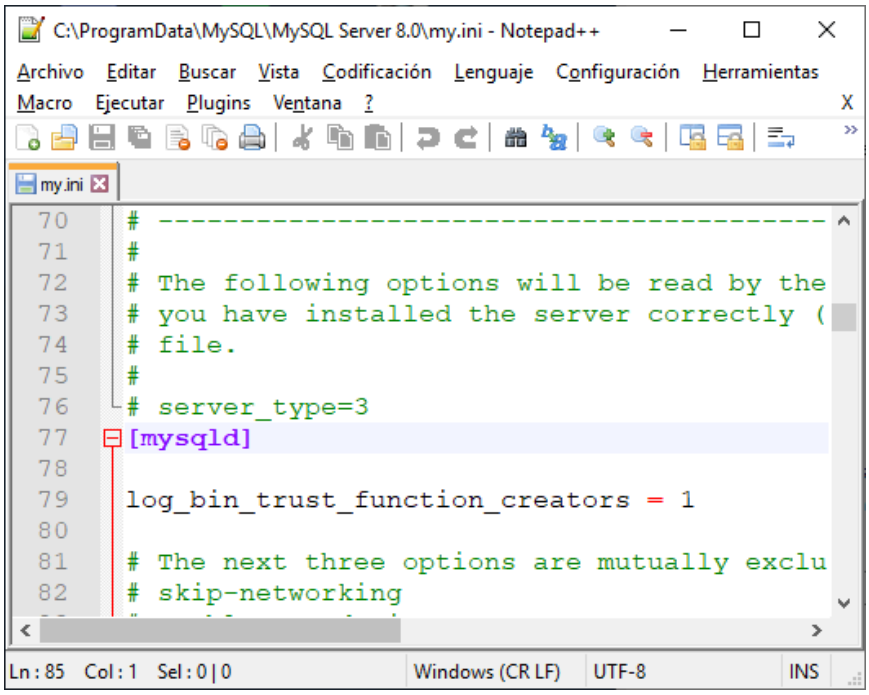


2) Cambiar la configuración para poder crear funciones sin restricciones de accesoa datos. Para ello editaremos el archivo de configuración:

`C:\ProgramData\MySQL\MySQL Server 8.0\my.ini`

Añadiremos en la sección **[mysqld]** la siguiente línea:

```
log_bin_trust_function_creators = 1
```



Después de estos dos cambios, deberemos reiniciar el servicio de **MySQL80** desde **services.msc**

Un ejemplo sencillo en MySQL de un procedimiento es:

```
/* Eliminar el procedimiento si ya existe */
DROP PROCEDURE IF EXISTS maximo;

/* Crear el procedimiento */
DELIMITER //
CREATE PROCEDURE maximo(IN num1 DECIMAL(11,2),
                        IN num2 DECIMAL(11,2),
                        OUT num3 DECIMAL(11,2))
  BEGIN
      IF (num1>num2) THEN
            SET num3 := num1;
      ELSE
            SET num3 := num2;
      END IF;

  END //
DELIMITER ;
```

A este tipo de funciones se les suele llamar también "**procedimientos almacenados**" (**Stored Procedures**) y se pueden llamar luego desde otros lenguajes en el equipo cliente. De hecho, es una de las formas de trabajar que se suelen recomendar para evitar ataques de "inyección de SQL".

```
CALL maximo(10.3, 21.4, @num);
SELECT @num;
```

De igual manera, las funciones (procedimientos almacenados) pueden acceder a datos de la BD.

En Java es posible crear un programa que llame a esa función, que le pase parámetros y que obtenga sus resultados.

Para ello, emplearemos un "**CallableStatement**":

JDBC05Procedure

```
public class JDBC05Procedure {
  public static void main(String[] args)
                    throws ClassNotFoundException, SQLException {

    // Conexión a la BD
    String url;
    url = "jdbc:mysql://localhost:3306/biblioteca";
    url +="?autoReconnect=true&useSSL=false&zeroDateTimeBehavior=convertToNull";
    url +="&serverTimezone=UTC";

    String usuario = "root";
    String password = "1234";

    Class.forName("com.mysql.cj.jdbc.Driver");
    Connection con = DriverManager.getConnection(url, usuario, password);

    // Execute
    CallableStatement cStmt = con.prepareCall("{call maximo(?,?,?)}");
    cStmt.setFloat(1, (float) 10.4);
    cStmt.setFloat(2, (float) 30.7);
    cStmt.registerOutParameter(3, Types.DECIMAL);

    cStmt.execute();
    float resultado = cStmt.getFloat(3);
    System.out.println("El máximo es: " + resultado);

    // Cerrar objetos
    cStmt.close();
    con.close();
  }
}
```

Si la función es más sencilla, es decir, que no tiene parámetros, quizá no se pueda llamar con un "**callable statement**", porque obtengamos un error diciendo que intentamos acceder a la columna 1 de 0 columnas disponibles en "**registerOut-Parameter**".

Esto no es grave, porque no se trataría de algo susceptible de ataques como los de inyección de SQL. Deberíamos realizar una función en vez de un procedimiento y se podría acceder simplemente con un SELECT.

Si creamos una función que devuelva el número de libros existentes:

```
/* Eliminar la función si ya existe */
DROP FUNCTION IF EXISTS totallibros;

/* Crear la función */
DELIMITER //
CREATE FUNCTION totallibros()
      RETURNS INT
  BEGIN
      DECLARE total INT;

      SELECT COUNT(*) INTO total FROM libros;

      RETURN total;
  END //
DELIMITER ;
```

Para llamarla basta con utilizar SELECT.

```
SELECT totallibros();
```

Ahora para llamarla utilizaremos un statement normal, como los de SELECT a la BD.

JDBC06Function

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;
import java.sql.ResultSet;

public class JDBC06Function {
  public static void main(String[] args)
                    throws ClassNotFoundException, SQLException {

    // Conexión a la BD
    String url;

    Class.forName("com.mysql.cj.jdbc.Driver");
    url = "jdbc:mysql://localhost:3306/biblioteca";
    url +="?autoReconnect=true&useSSL=false&zeroDateTimeBehavior=convertToNull";
    url +="&serverTimezone=UTC";
    String usuario = "root";
    String password = "1234";
    Connection con = DriverManager.getConnection(url, usuario, password);

    // Execute
    Statement statement = con.createStatement();
    String consulta = "SELECT totallibros()";

    ResultSet rs = statement.executeQuery(consulta);
    rs.next();
    System.out.println("El total de libros es: " + rs.getInt(1));

    // Cerrar objetos
    rs.close();
    statement.close();
    con.close();
  }
}
```

9. Gestión de errores

Hasta ahora, no estamos tratando los errores de ninguna forma, simplemente estamos dejando que "**main**" se interrumpa lanzando la correspondiente excepción;

```
  public static void main(String[] args)
                    throws ClassNotFoundException, SQLException {
```

Una alternativa para gestionar los errores que además nos cierre las conexiones es usar un bloque **try-catch** para atrapar el error, informar al usuario y permitir que el programa prosiga a pesar del problema.

JDBC07SelectTry1

```
import java.sql.*;

public class JDBC07SelectTry1 {
 public static void main(String[] args) {
  try {

    Class.forName("com.mysql.cj.jdbc.Driver");

    // Conexión a la BD
    String url;
    url = "jdbc:mysql://localhost:3306/biblioteca";
    url +="?autoReconnect=true&useSSL=false&zeroDateTimeBehavior=convertToNull";
    url +="&serverTimezone=UTC";

    String usuario = "root";
    String password = "1234";

    String sentenciaSQL = "SELECT titulo, autor FROM libros";

    try (  Connection con = DriverManager.getConnection(url, usuario, password);
           Statement statement = con.createStatement();
           ResultSet rs = statement.executeQuery(sentenciaSQL);
          ) {
      System.out.printf("%-60s %-60s\n", "Título", "Autor");
      System.out.print("------------------------------------------------------------");
      System.out.print(" ");
      System.out.print("------------------------------------------------------------");
      System.out.println();
      while (rs.next()) {
        System.out.printf("%-60s %-60s\n",
                          rs.getString("titulo"),
                          rs.getString("autor"));
      }
      System.out.println("----------");
      System.out.println("Terminado!");

    } catch (SQLException se) {
      se.printStackTrace();
    }
  } catch (ClassNotFoundException ce) {
    ce.printStackTrace();
  }
 }
}
```

Como se puede ver, aparece el mensaje "Terminado!" al final, lo que indica que la ejecución no se ha interrumpido aunque se muestre algún error.

Aun así, en general, será preferible analizar los posibles errores por separado, de modo que los mensajes de error (y la posible recuperación de éstos) sean más específicos:

```
...

      System.out.println("----------");
      System.out.println("Terminado!");

    } catch (SQLException se) {
      // se.printStackTrace();
      System.out.println("No se ha podido realizar el SELECT");
    }
  } catch (ClassNotFoundException ce) {
    // ce.printStackTrace();
    System.out.println("MySQL no accesible");
  }
 }

}
```

De hecho, incluso una "**SQLException**" es algo demasiado genérico, de modo que en un caso real se podrían analizar los valores de "**getMessage()**" y de "**getErrorCode()**" para saber el tipo de error concreto que ha existido. En general, esto puede ser preferible a usar varios bloques try-catch.

JDBC07SelectTry2

```
...
      System.out.println("----------");
      System.out.println("Terminado!");

    } catch (SQLException se) {
      // se.printStackTrace();
      System.out.println("No se ha podido realizar el SELECT");
      switch (se.getErrorCode()) {
         case 1054:
            // Columna desconocida
            System.out.println("ERROR: Columna desconocida");
            System.out.println(se.getErrorCode()+" "+se.getMessage());
            break;

         case 1064:
            // Error de sintaxis
            System.out.println("ERROR de SINTAXIS");
            System.out.println(se.getErrorCode()+" "+se.getMessage());
            break;

         default:
            System.out.println(se.getErrorCode()+" "+se.getMessage());
      }
    }
  } catch (ClassNotFoundException ce) {
    // ce.printStackTrace();
    System.out.println("MySQL no accesible");
  }
 }
}
```

Prueba la ejecución de **JDBC07SelectTry2** con algún error en la sentencia **SELECT** como:

- `SELECT titulo, autor FOM libros`
- `SELECT titlo, autor FROM libros`

Para consultar los códigos de errores generados en MySQL podemos consultar el siguiente enlace.

MySQL Reference - Códigos de errores y sus mensajes
https://dev.mysql.com/doc/mysql-errors/8.0/en/server-error-reference.html

Otro ejemplo sería simplemente mostrar unos errores usando la clase "**Logger**" y otros mostrando un mensaje específico**.**

Por ejemplo, podemos crear una versión alternativa del programa **JDBC02Create**, que creaba una tabla, pero añadiendo esta vez la posibilidad de que avise en caso de que no se haya podido crear la tabla (cosa que ocurrirá en esta segunda ocasión, en que la tabla ya existe).

JDBC08CreateTry

```
public class JDBC02CreateTry {
  public static void main(String[] args) {
    try {
      Class.forName("com.mysql.cj.jdbc.Driver");

      // Conexión a la BD
      String url;
      url = "jdbc:mysql://localhost:3306/biblioteca";
      url +="?autoReconnect=true&useSSL=false&zeroDateTimeBehavior=convertToNull";
      url +="&serverTimezone=UTC";

      String usuario = "root";
      String password = "1234";

      // Crear Statement del CREATE TABLE
      String sentenciaSQL = "CREATE TABLE personas ( "+
                "codigo VARCHAR(4) PRIMARY KEY, "+
                "nombre VARCHAR(50), "+
                "email VARCHAR(40) "+ ");";

      try (  Connection con = DriverManager.getConnection(url, usuario, password);
             Statement statement = con.createStatement(); ) {

         // Execute
         statement.executeUpdate(sentenciaSQL);
         System.out.println("Tabla personas creada");

      } catch (SQLException ex) {

        System.out.println("Error en sentencia CREATE de SQL.");
        System.out.println(ex.getErrorCode()+" "+ex.getMessage());

      }
    } catch (ClassNotFoundException ex) {
      Logger.getLogger(JDBC08CreateTry.class.getName()).log(Level.SEVERE, null, ex);
    }
  }
}
```

Si deseamos obtener los mensajes de error en español, deberemos cambiar la configuración por defecto de MySQL. Para ello editaremos el archivo de configuración **my.ini** de:

C:\ProgramData\MySQL\MySQL Server 8.0\my.ini

y añadiremos en la sección **[mysqld]** la entrada de la variable **lc_messages**:

```
[mysqld]

lc_messages=es_ES
```

Después de realizar el cambio deberemos reiniciar el servicio de mysql denominado MySQL80.

A continuación, realizaremos un proyecto similar al ejercicio **Fich19SerializacionPersonaOptimizado** denominado **UD3MysqlPlantilla** que utilice MySQL en vez de ficheros binarios.

UD3MysqlPlantilla

El proyecto mostrará un menú, pero antes conectará con MySQL y después desconectará de MySQL.

La ejecución será:

```
Conectado a MySQL
----------------------------------------
1. Mostrar todos los libros
2. Añadir datos de libros
3. Buscar libro por autor
--------------------
0. Salir
--------------------
Escribe una de las opciones: 1
----------------------------------------
+----+-------------------------------------------------+------------------------------+
| ID | Título                                          | Autor                        |
+----+-------------------------------------------------+------------------------------+
|  1 | Macbeth                                         | William Shakespeare          |
|  2 | La Celestina (Tragicomedia de Calisto y Melibea)| Fernando de Rojas            |
|  3 | El Lazarillo de Tormes                          | Anónimo                      |
|  4 | 20.000 Leguas de Viaje Submarino                | Julio Verne                  |
|  5 | Alicia en el País de las Maravillas             | Lewis Carrol                 |
|  6 | Cien Años de Soledad                            | Gabriel García Márquez       |
|  7 | La tempestad                                    | William Shakespeare          |
|  8 | La metamorfosis                                 | Franz Kafka                  |
|  9 | Romeo y Julieta                                 | William Shakespeare          |
+----+-------------------------------------------------+------------------------------+

----------------------------------------
1. Mostrar todos los libros
2. Añadir datos de libros
3. Buscar libro por autor
--------------------
0. Salir
--------------------
Escribe una de las opciones: 0
----------------------------------------
Terminado
----------------------------------------
Desconectado de MySQL
```

Puedes probar y comprobar el funcionamiento del proyecto entregado por el profesor.